

5.6 - Blue/green e rolling deployment: troca de versão com controle

Duas formas de trocar de versão sem derrubar quem está operando agora

Deploy crítico às 14h de uma sexta-feira

- **Mudança crítica no serviço de tracking, transportadoras operando em pico de demanda**
 - Janela de manutenção planejada é inviável, operações não param às 14h

LUNALOG

A LunaLog precisava implantar uma correção urgente no serviço de tracking, e o único horário disponível era 14h de uma sexta-feira, o pico absoluto de operações das transportadoras. Um rolling deployment causaria janela de instabilidade enquanto instâncias antigas e novas coexistissem, e algumas requisições poderiam ver respostas inconsistentes durante essa transição. Ana defendeu blue/green, dois ambientes em paralelo, validação completa do novo antes de redirecionar uma única requisição real. Roberto aprovou. O deploy aconteceu sem que nenhuma transportadora percebesse, o ambiente verde foi promovido para azul em uma chamada de API ao load balancer.

Blue/green deployment, dois ambientes em paralelo

Troca instantânea de tráfego, rollback em uma chamada

- **Dois ambientes idênticos em paralelo, apenas um recebe tráfego real por vez**
 - Azul é o ambiente atual em produção, verde é a nova versão sendo preparada
- **Nova versão é validada completamente no verde antes de qualquer tráfego real**
 - Smoke tests, validação manual, integrações ponta a ponta, tudo antes do switch
- **Promoção é uma chamada ao load balancer ou DNS, segundos de transição**
 - Rollback é igualmente instantâneo, apontar de volta para o ambiente anterior
- **Custo é alto, dois ambientes plenamente provisionados simultaneamente**
 - Adequado para serviços críticos com tolerância zero a inconsistência durante deploy

Rolling deployment, substituição gradual

Trocar instâncias uma de cada vez, sem ambiente duplicado

- **Instâncias antigas são substituídas progressivamente por instâncias novas**
 - Uma a uma, ou em lotes pequenos, definidos pelo orquestrador
- **Durante a transição, instâncias antigas e novas coexistem servindo tráfego**
 - Cada requisição pode cair em qualquer versão durante a janela de rollout
- **Exige compatibilidade entre versões adjacentes da aplicação**
 - Schema de banco, formato de cache, estrutura de mensagens, tudo precisa coexistir
- **Custo de infraestrutura é baixo, sem necessidade de ambiente duplicado**
- **Rollback exige rolling reverso, mais lento que blue/green**
 - Padrão default em Kubernetes Deployments com strategy RollingUpdate

Blue/green contra rolling, dimensões de comparação

Blue/green

- Custo de infraestrutura, dois ambientes em paralelo
- Velocidade de rollback, segundos, switch reverso
- Estado durante deploy, totalmente consistente
- Compatibilidade entre versões, não exigida
- Adequado para mudanças incompatíveis com versão atual
- Complexidade operacional, sincronização de estado

Rolling deployment

- Custo de infraestrutura, baixo, sem duplicação
- Velocidade de rollback, minutos, rolling reverso
- Estado durante deploy, mistura de versões coexistindo
- Compatibilidade entre versões, obrigatória
- Adequado para mudanças backward compatible
- Complexidade operacional, gestão de health checks

A pegadinha do estado durante a transição

- **Rolling exige que versões adjacentes coexistam sem conflito de dados**

Em rolling deployment, se a nova versão alterar formato de cache, schema de mensageria ou estrutura de sessão, requisições servidas pela versão antiga vão falhar ou corromper dados durante a janela de transição. A única forma segura é garantir que cada deploy faça uma única mudança incremental, sempre backward compatible com a versão anterior.

- Esse é exatamente o problema que o padrão expand-contract resolve, voltaremos na aula 5.8



Blue/green e rolling cobrem trocas controladas de versão, mas e quando você quer expor uma nova funcionalidade para apenas cinco por cento dos usuários antes de abrir para todos, sem fazer um deploy diferente para cada grupo? Existe um par de técnicas que desacopla o ato de implantar o código do ato de liberar a funcionalidade.

Canary release e feature flags, exposição progressiva e desacoplamento